Lean

PENG

November 2, 2026

Contents

1 Introduction **1**

2 Simple Type Theory **1**

3 Dependent Type Theory **2**

4 Tactics **2**

A Tools **2**

1 Introduction

Definition 1.1 (lean)**.** *Lean is an expression-oriented functional language based on dependent type theory.*

2 Simple Type Theory

`https://lean-lang.org/theorem_proving_in_lean4/`

Definition 2.1 (declare object)**.** *Using* `def` *to declare a new constant symbol.*

```
def obj : type := value
#check obj -- check type
#eval expr -- evaluate
```

Definition 2.2 (types)**.** *Every expression has an associated type.*

```
Nat
Int
Float
String
Bool -- True or False
List
Type
```

Definition 2.3 (function)**.**

```
def func (param_1 : type) ... (param_n : type) : type :=
    body

-- call function
func arg_1 ... arg_n
```

Definition 2.4 (simple type theory)**.** *Build new types out of others.*

```
type → type -- denotes the type of function from type to type
type → type → type -- equivalent to type → (type → type) that
-- takes type and return another function that takes type and return type

type × type -- denotes the type of pairs consisting of type paired with type
(m, n) denotes the ordered pair of m and n which is of type × type
(m, n).1 : type
(m, n).2 : type
```

3 Dependent Type Theory

Definition 3.1 (dependent type theory)**.** *Types are objects.*

Definition 3.2 (declare type)**.**

```
def T : Type := type
```

Definition 3.3 (proposition)**.**

Definition 3.4 (theorem)**.**

4 Tactics

Tactic 4.1 (reflexivity of equality)**.** `ref` *proves goals of the form* `X = X`*.*

Tactic 4.2 (rewrite)**.** *If* `h` *is a proof of an equality* `X = Y`*, then* `rw [h]` *replace all* `X` *with* `Y`*.*

```
repeat rw [h] -- rewrite at once
rw [← h] -- turn Y into X
```

Definition 4.1 ($\mathbb{N}$)**.** $\mathbb{N}$ *is the natural numbers defined by* `0 : ℕ` *and* `succ (0 : ℕ) : ℕ`*.*

Theorem 4.1 (successor)**.** *The successor of* `n` *means the number after* `n`*.*

```
two_eq_succ_one -- proof of 2 = succ 1
add_succ x d : x + succ d = succ (x + d)
```

Theorem 4.2 (adding 0)**.** *Define* `a + 0` *to be* `a` *for any number* `a`*.*

```
add_zero   -- proof of ? + 0 = ?
add_zero x -- proof of x + 0 = x
```

A Tools

Definition A.1 (comment)**.**

```
/- comment blocks can be nested -/
-- line comment
```